ASSIGNMENT_8_0733_MDTS4214

ROUNAK SENGUPTA

2026-03-30

Problem: Logistic Regression with Binary Response

We use the `Weekly` data from the `ISLR` package to demonstrate logistic regression for a binary response variable.

(a) Train-test split

We use observations from **1990 to 2008** as the training data and observations from **2009 to 2010** as the test data.

```
library(ISLR)
data("Weekly")

train_data <- subset(Weekly, Year >= 1990 & Year <= 2008)
test_data  <- subset(Weekly, Year >= 2009 & Year <= 2010)

dim(train_data)

## [1] 985   9

dim(test_data)

## [1] 104   9
```

The response variable is `Direction`, and the predictors are `Lag1`, `Lag2`, `Lag3`, `Lag4`, `Lag5`, and `Volume`.

(b) Logistic regression with five lag variables and Volume

We now fit a logistic regression model on the training data.

```
glm_full <- glm(Direction ~ Lag1 + Lag2 + Lag3 + Lag4 + Lag5 + Volume,
                data = train_data,
                family = binomial)

summary(glm_full)

##
## Call:
## glm(formula = Direction ~ Lag1 + Lag2 + Lag3 + Lag4 + Lag5 +
##     Volume, family = binomial, data = train_data)
##
```

```
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept)  0.33258    0.09421   3.530 0.000415 ***
## Lag1        -0.06231    0.02935  -2.123 0.033762 *
## Lag2         0.04468    0.02982   1.499 0.134002
## Lag3        -0.01546    0.02948  -0.524 0.599933
## Lag4        -0.03111    0.02924  -1.064 0.287241
## Lag5        -0.03775    0.02924  -1.291 0.196774
## Volume      -0.08972    0.05410  -1.658 0.097240 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 1354.7  on 984  degrees of freedom
## Residual deviance: 1342.3  on 978  degrees of freedom
## AIC: 1356.3
##
## Number of Fisher Scoring iterations: 4
```

Significance of predictors

The p-values of the predictors are:

```
full_summary <- summary(glm_full)
coef_table <- full_summary$coefficients
coef_table

##                Estimate Std. Error    z value     Pr(>|z|)
## (Intercept)  0.33258372 0.09420696  3.5303518 0.0004150074
## Lag1        -0.06231372 0.02935315 -2.1228973 0.0337624608
## Lag2         0.04467963 0.02981610  1.4985069 0.1340016131
## Lag3        -0.01545955 0.02947503 -0.5244964 0.5999333317
## Lag4        -0.03111238 0.02923566 -1.0641929 0.2872413213
## Lag5        -0.03774640 0.02924268 -1.2907983 0.1967736430
## Volume      -0.08971724 0.05409925 -1.6583824 0.0972403108
```

We now identify whether any predictor is significant at the 5% level.

```
pvals <- coef_table[-1, 4]   # exclude intercept
significant_predictors <- names(pvals[pvals < 0.05])
significant_predictors

## [1] "Lag1"
```

If no predictor is significant at the 5% level, then we find the predictor with the smallest p-value.

```
least_p_name <- names(which.min(pvals))
least_p_value <- min(pvals)
```

```
least_p_name

## [1] "Lag1"

least_p_value

## [1] 0.03376246
```

Interpretation of the coefficient with the least p-value

The corresponding coefficient is:

```
least_p_coef <- coef(glm_full)[least_p_name]
least_p_coef

##        Lag1
## -0.06231372
```

Its odds ratio is:

```
exp(least_p_coef)

##      Lag1
## 0.9395881
```

Interpretation

For a one-unit increase in Lag1, keeping all other predictors fixed, the **log-odds** of the market going **Up** changes by -0.0623.

Equivalently, the **odds** are multiplied by 0.9396.

- If this value is greater than 1, the odds of market going **Up** increase.
- If this value is less than 1, the odds of market going **Up** decrease.

(c) Odds of market going "Up" at median predictor values

We compute the median of each predictor using the training data.

```
med_vals <- data.frame(
  Lag1 = median(train_data$Lag1),
  Lag2 = median(train_data$Lag2),
  Lag3 = median(train_data$Lag3),
  Lag4 = median(train_data$Lag4),
  Lag5 = median(train_data$Lag5),
  Volume = median(train_data$Volume)
)

med_vals
```

```
##    Lag1  Lag2  Lag3 Lag4 Lag5   Volume
## 1 0.231 0.234 0.231 0.23 0.23 0.804848
```

Now we estimate the probability of the market going **Up** at these median predictor values.

```
p_up_median <- predict(glm_full, newdata = med_vals, type = "response")
p_up_median

##         1
## 0.5589808
```

The estimated **odds** are:

```
odds_up_median <- p_up_median / (1 - p_up_median)
odds_up_median

##        1
## 1.267475
```

Thus, at the median values of the predictors, the estimated odds that the market goes **Up** are 1.2675.

(e) Optimal cut-point on the test data

We now use the fitted model to obtain predicted probabilities for the test data.

```
test_prob_full <- predict(glm_full, newdata = test_data, type = "response")
head(test_prob_full)

##       986       987       988       989       990       991
## 0.3900305 0.6084798 0.4489399 0.4101072 0.4336596 0.4406039
```

We define a function to compute:

- True Positive Rate (TPR)
- False Positive Rate (FPR)
- Objective: `TPR * (1 - FPR)`

for a given cut-point.

```
evaluate_cutpoint <- function(actual, prob, cutoff) {
  pred <- ifelse(prob > cutoff, "Up", "Down")
  pred <- factor(pred, levels = c("Down", "Up"))
  actual <- factor(actual, levels = c("Down", "Up"))

  cm <- table(Predicted = pred, Actual = actual)

  # make sure all cells exist
  if (!("Down" %in% rownames(cm))) {
    cm <- rbind(cm, Down = c(0, 0))
```

```r
  }
  if (!("Up" %in% rownames(cm))) {
    cm <- rbind(cm, Up = c(0, 0))
  }
  if (!("Down" %in% colnames(cm))) {
    cm <- cbind(cm, Down = c(0, 0))
  }
  if (!("Up" %in% colnames(cm))) {
    cm <- cbind(cm, Up = c(0, 0))
  }

  cm <- cm[c("Down", "Up"), c("Down", "Up")]

  TN <- cm["Down", "Down"]
  FN <- cm["Down", "Up"]
  FP <- cm["Up", "Down"]
  TP <- cm["Up", "Up"]

  TPR <- ifelse((TP + FN) == 0, 0, TP / (TP + FN))
  FPR <- ifelse((FP + TN) == 0, 0, FP / (FP + TN))
  objective <- TPR * (1 - FPR)
  error_rate <- (FP + FN) / sum(cm)

  list(
    cutoff = cutoff,
    confusion_matrix = cm,
    TPR = TPR,
    FPR = FPR,
    objective = objective,
    error_rate = error_rate
  )
}
```

Now we search over possible cut-points.

```r
cut_grid <- sort(unique(test_prob_full))
results_full <- lapply(cut_grid, function(cut) {
  evaluate_cutpoint(test_data$Direction, test_prob_full, cut)
})

objective_values_full <- sapply(results_full, function(x) x$objective)
best_index_full <- which.max(objective_values_full)
best_result_full <- results_full[[best_index_full]]

best_result_full$cutoff

## [1] 0.4556775

best_result_full$TPR
```

```
## [1] 0.557377
```

```r
best_result_full$FPR
```

```
## [1] 0.4883721
```

```r
best_result_full$objective
```

```
## [1] 0.2851697
```

Confusion matrix for the optimal cut-point

```r
best_result_full$confusion_matrix
```

```
##          Actual
## Predicted Down Up
##      Down   22 27
##      Up     21 34
```

Test error rate

```r
best_result_full$error_rate
```

```
## [1] 0.4615385
```

So, using the optimal cut-point, the test error rate is 0.4615.

(f) Logistic regression using only Lag1 and Lag2

Now we fit another model using only `Lag1` and `Lag2`.

```r
glm_lag12 <- glm(Direction ~ Lag1 + Lag2,
                 data = train_data,
                 family = binomial)

summary(glm_lag12)
```

```
##
## Call:
## glm(formula = Direction ~ Lag1 + Lag2, family = binomial, data =
train_data)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept)  0.21109    0.06456   3.269  0.00108 **
## Lag1        -0.05421    0.02886  -1.878  0.06034 .
## Lag2         0.05384    0.02905   1.854  0.06379 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
```

```
##
##     Null deviance: 1354.7  on 984  degrees of freedom
## Residual deviance: 1347.0  on 982  degrees of freedom
## AIC: 1353
##
## Number of Fisher Scoring iterations: 4
```

We obtain predicted probabilities for the test data.

```
test_prob_lag12 <- predict(glm_lag12, newdata = test_data, type = "response")
head(test_prob_lag12)
```

```
##       986       987       988       989       990       991
## 0.4386143 0.6934260 0.5539158 0.5209042 0.5338470 0.4728669
```

Again, we find the optimal cut-point by maximizing `TPR * (1 - FPR)`.

```
cut_grid_lag12 <- sort(unique(test_prob_lag12))
results_lag12 <- lapply(cut_grid_lag12, function(cut) {
  evaluate_cutpoint(test_data$Direction, test_prob_lag12, cut)
})

objective_values_lag12 <- sapply(results_lag12, function(x) x$objective)
best_index_lag12 <- which.max(objective_values_lag12)
best_result_lag12 <- results_lag12[[best_index_lag12]]

best_result_lag12$cutoff
```

```
## [1] 0.5268024
```

```
best_result_lag12$TPR
```

```
## [1] 0.7868852
```

```
best_result_lag12$FPR
```

```
## [1] 0.5581395
```

```
best_result_lag12$objective
```

```
## [1] 0.3476935
```

Confusion matrix for the Lag1-Lag2 model

```
best_result_lag12$confusion_matrix
```

```
##          Actual
## Predicted Down Up
##      Down   19 13
##      Up     24 48
```

Test error rate for the Lag1-Lag2 model

```
best_result_lag12$error_rate

## [1] 0.3557692
```

Comparison of test error rates

```
full_error <- best_result_full$error_rate
lag12_error <- best_result_lag12$error_rate

comparison <- data.frame(
  Model = c("Lag1 + Lag2 + Lag3 + Lag4 + Lag5 + Volume", "Lag1 + Lag2"),
  Test_Error_Rate = c(full_error, lag12_error)
)

comparison

##                                          Model Test_Error_Rate
## 1 Lag1 + Lag2 + Lag3 + Lag4 + Lag5 + Volume       0.4615385
## 2                                 Lag1 + Lag2       0.3557692
```

Conclusion

```
if (lag12_error < full_error) {
  cat("The model with only Lag1 and Lag2 gives a lower test error rate.")
} else if (lag12_error > full_error) {
  cat("The model with all six predictors gives a lower test error rate.")
} else {
  cat("Both models give the same test error rate.")
}

## The model with only Lag1 and Lag2 gives a lower test error rate.
```